leanprover-community.github.io

Mathlib naming conventions

17–22 minutes

This guide is written for Lean 4.

File names

`.lean` files in mathlib should generally be named in `UpperCamelCase`. A (very rare) exception are files named after some specifically lower-cased object, e.g. `lp.lean` for a file specifically about the space (and not). Such exceptions should be discussed on Zulip first.

General conventions

Capitalization

Unlike Lean 3, in which the convention was that all declarations used `snake_case`, in mathlib under Lean 4 we use a combination of `snake_case`, `lowerCamelCase` and `UpperCamelCase` according to the following naming scheme.

1. Terms of `Prop`s (e.g. proofs, theorem names) use `snake_case`.
2. `Prop`s and `Type`s (or `Sort`) (inductive types, structures, classes) are in `UpperCamelCase`. There are some rare exceptions: some fields of structures are currently wrongly lower-cased (see the example for the class `LT` below).
3. Functions are named the same way as their return values (e.g. a function of type `A → B → C` is named as though it is a term of type `C`).
4. All other terms of `Type`s (basically anything else) are in `lowerCamelCase`.
5. When something named with `UpperCamelCase` is part of something named with `snake_case`, it is referenced in `lowerCamelCase`.
6. Acronyms like `LE` are written upper-/lowercase as a group, depending on what the first character would be.
7. Rules 1-6 apply to fields of a structure or constructors of an inductive type in the same way.

   There are some rare exceptions to preserve local naming symmetry: e.g., we use `Ne` rather than `NE` to follow the example of `Eq`; `outParam` has a `Sort` output but is not `UpperCamelCase`. Some other exceptions include intervals (`Set.Icc`, `Set.Iic`, etc.), where the `I` is capitalized despite the fact that it should be `lowerCamelCase` according to the convention. Any such exceptions should be discussed on Zulip.

Examples

```
-- follows rule 2
structure OneHom (M : Type _) (N : Type _) [One M] [One N] where
  toFun : M → N -- follows rule 4 via rule 3 and rule 7
  map_one' : toFun 1 = 1 -- follows rule 1 via rule 7
```

```
-- follows rule 2 via rule 3
class CoeIsOneHom [One M] [One N] : Prop where
  coe_one : (↑(1 : M) : N) = 1 -- follows rule 1 via rule 6

-- follows rule 1 via rule 3
theorem map_one [OneHomClass F M N] (f : F) : f 1 = 1 := sorry

-- follows rules 1 and 5
theorem MonoidHom.toOneHom_injective [MulOneClass M] [MulOneClass N] :
  Function.Injective (MonoidHom.toOneHom : (M →* N) → OneHom M N) := sorry

-- follows rule 2
class HPow (α : Type u) (β : Type v) (γ : Type w) where
  hPow : α → β → γ -- follows rule 3 via rule 6; note that rule 5 does not
apply

-- follows rules 2 and 6
class LT (α : Type u) where
  lt : α → α → Prop -- this is an exception to rule 2

-- follows rules 2 (for `Semifield`) and 4 (for `toIsField`)
theorem Semifield.toIsField (R : Type u) [Semifield R] :
    IsField R -- follows rule 2

-- follows rules 1 and 6
theorem gt_iff_lt [LT α] {a b : α} : a > b ↔ b < a := sorry

-- follows rule 2; `Ne` is an exception to rule 6
class NeZero : Prop := sorry

-- follows rules 1 and 5
theorem neZero_iff {R : Type _} [Zero R] {n : R} : NeZero n ↔ n ≠ 0 := sorry
```

Spelling #

Declaration names use American English spelling. So e.g. we use `factorization`, `Localization` and `FiberBundle` and not `factorisation`, `Localisation` or `FibreBundle`. Contrast this with the rule for documentation, which is allowed to use other common English spellings.

Names of symbols #

When translating the statements of theorems into words, the following dictionary is often used.

Logic #

symbol	shortcut	name	notes
∨	\or	or	
∧	\and	and	
→	\r	of / imp	the conclusion is stated first and the hypotheses are often omitted

symbol	shortcut	name	notes
↔	\iff	iff	sometimes omitted along with the right hand side of the iff
¬	\n	not	
∃	\ex	exists / bex	bex stands for "bounded exists"
∀	\fo	all / forall / ball	ball stands for "bounded forall"
=		eq	often omitted
≠	\ne	ne	
∘	\o	comp	

ball and bex are still used in Lean core, but should not be used in mathlib.

Set #

symbol	shortcut	name	notes
∈	\in	mem	
∉	\notin	notMem	
∪	\cup	union	
∩	\cap	inter	
⋃	\bigcup	iUnion / biUnion	i for "indexed", bi for "bounded indexed"
⋂	\bigcap	iInter / biInter	i for "indexed", bi for "bounded indexed"
$⋃_0$	\bigcup\0	sUnion	s for "set"
$⋂_0$	\bigcap\0	sInter	s for "set"
\	\\	sdiff	
ᶜ	\^c	compl	
{x | p x}		setOf	
{x}		singleton	
{x, y}		pair	

Algebra #

symbol	shortcut	name	notes
0		zero	
+		add	
-		neg / sub	neg for the unary function, sub for the binary function
1		one	

symbol	shortcut	name	notes
*		mul	
^		pow	
/		div	
•	\bu	smul	
⁻¹	\-1	inv	
⁻¹ᐟ	\frac1	invOf	
∣	\\|	dvd	
∑	\sum	sum	
∏	\prod	prod	

Lattices #

symbol	shortcut	name	notes
<		lt / gt	
≤	\le	le / ge	
⊔	\sup	sup	a binary operator
⊓	\inf	inf	a binary operator
⨆	\supr	iSup / biSup / ciSup	c for "conditionally complete"
⨅	\infi	iInf / biInf / ciInf	c for "conditionally complete"
⊥	\bot	bot	
⊤	\top	top	

The symbols ≤ and < have a special naming convention. In mathlib, we almost always use ≤ and < instead of ≥ and >, so we can use both `le`/`lt` and ge/gt for naming ≤ and <. There are a few reasons to use ge/gt:

1. We use ge/gt if the arguments to ≤ or < appear in different orders. We use `le`/`lt` for the first occurrence of ≤/< in the theorem name, and then ge/gt indicates that the arguments are swapped.
2. We use ge/gt to match the argument order of another relation, such as = or ≠.
3. We use ge/gt to describe the ≤ or < relation with its arguments swapped.
4. We use ge/gt if the second argument to ≤ or < is 'more variable'.

```
-- follows rule 1
theorem lt_iff_le_not_ge [Preorder α] {a b : α} : a < b ↔ a ≤ b ∧ ¬b ≤ a :=
sorry
theorem not_le_of_gt [Preorder α] {a b : α} (h : a < b) : ¬b ≤ a := sorry
theorem LT.lt.not_ge [Preorder α] {a b : α} (h : a < b) : ¬b ≤ a := sorry

-- follows rule 2
theorem Eq.ge [Preorder α] {a b : α} (h : a = b) : b ≤ a := sorry
```

```
theorem ne_of_gt [Preorder α] {a b : α} (h : b < a) : a ≠ b := sorry

-- follows rule 3
theorem ge_trans [Preorder α] {a b : α} : b ≤ a → c ≤ b → c ≤ a := sorry

-- follows rule 4
theorem le_of_forall_gt [LinearOrder α] {a b : α} (H : ∀ (c : α), a < c → b <
c) : b ≤ a := sorry
```

Coercions #

Coercions are named after the underlying function.

```
-- Named after `Subtype.val`
theorem Subtype.val_injective {p : α → Prop} : ((↑) : {a : α // p a} →
α).Injective := sorry

-- Named after `ENNReal.ofNNReal`
theorem ENNReal.ofNNReal_injective : ((↑) : ℝ≥0 → ℝ≥0∞).Injective := sorry

-- Named after `DFunLike.coe`
theorem DFunLike.coe_injective {F α : Sort*} {β : α → Sort*} [DFunLike F α β] :
    ((↑) : F → ∀ a, β a).Injective := sorry

-- Named after `SetLike.coe`
theorem SetLike.coe_injective {α β : Type*} [SetLike α β] :
    ((↑) : α → Set β).Injective := sorry
```

This helps to disambiguate when types support several natural coercions, and is motivated by the fact that coercions are reducible. This wasn't the case in Lean 3, and therefore many names are wrong still.

Dots #

Dots are used for namespaces, and also for automatically generated names like recursors, eliminators and structure projections. They can also be introduced manually, for example, where projector notation is useful. Thus they are used in all of the following situations.

Note: since And is a (binary function into) Prop, it is UpperCamelCased according to the naming conventions, and so its namespace is And . *. This may seem at odds with the dictionary ∧ --> and but because upper camel case types get lower camel cased when they appear in names of theorems, the dictionary is still valid in general. The same applies to Or, Iff, Not, Eq, HEq, Ne, etc.

Intro, elim, and destruct rules for logical connectives, whether they are automatically generated or not:

- And.intro
- And.elim
- And.left
- And.right
- Or.inl
- Or.inr
- Or.intro_left

- `Or.intro_right`
- `Iff.intro`
- `Iff.elim`
- `Iff.mp`
- `Iff.mpr`
- `Not.intro`
- `Not.elim`
- `Eq.refl`
- `Eq.rec`
- `Eq.subst`
- `HEq.refl`
- `HEq.rec`
- `HEq.subst`
- `Exists.intro`
- `Exists.elim`
- `True.intro`
- `False.elim`

Places where projection notation is useful, for example:

- `And.symm`
- `Or.symm`
- `Or.resolve_left`
- `Or.resolve_right`
- `Eq.symm`
- `Eq.trans`
- `HEq.symm`
- `HEq.trans`
- `Iff.symm`
- `Iff.refl`

It is useful to use dot notation even for types which are not inductive types. For instance, we use:

- `LE.trans`
- `LT.trans_le`
- `LE.trans_lt`

Axiomatic descriptions #

Some theorems are described using axiomatic names, rather than describing their conclusions.

- `def` (for unfolding a definition)

- `refl`
- `irrefl`
- `symm`
- `trans`
- `antisymm`
- `asymm`
- `congr`
- `comm`
- `assoc`
- `left_comm`
- `right_comm`
- `mul_left_cancel`
- `mul_right_cancel`
- `inj` (injective)

Variable conventions #

- u, v, w, ... for universes
- α, β, γ, ... for generic types
- a, b, c, ... for propositions
- x, y, z, ... for elements of a generic type
- h, h_1, ... for assumptions
- p, q, r, ... for predicates and relations
- s, t, ... for lists
- s, t, ... for sets
- m, n, k, ... for natural numbers
- i, j, k, ... for integers

Types with a mathematical content are expressed with the usual mathematical notation, often with an upper case letter (G for a group, R for a ring, K or 𝕜 for a field, E for a vector space, ...). This convention is not followed in older files, where greek letters are used for all types. Pull requests renaming type variables in these files are welcome.

Identifiers and theorem names

We adopt the following naming guidelines to make it easier for users to guess the name of a theorem or find it using tab completion. Common "axiomatic" properties of an operation like conjunction or disjunction are put in a namespace that begins with the name of the operation:

```
import Mathlib.Logic.Basic

#check And.comm
#check Or.comm
```

In particular, this includes `intro` and `elim` operations for logical connectives, and properties of relations:

```
import Mathlib.Logic.Basic

#check And.intro
#check And.elim
#check Or.intro_left
#check Or.intro_right
#check Or.elim

#check Eq.refl
#check Eq.symm
#check Eq.trans
```

Note however we do not do this for axiomatic logical and arithmetic operations.

```
import Mathlib.Algebra.Group.Basic

#check and_assoc
#check mul_comm
#check mul_assoc
#check @mul_left_cancel  -- multiplication is left cancelative
```

For the most part, however, we rely on descriptive names. Often the name of theorem simply describes the conclusion:

```
import Mathlib.Algebra.Ring.Basic
open Nat
#check succ_ne_zero
#check mul_zero
#check mul_one
#check @sub_add_eq_add_sub
#check @le_iff_lt_or_eq
```

If only a prefix of the description is enough to convey the meaning, the name may be made even shorter:

```
import Mathlib.Algebra.Ring.Basic

#check @neg_neg
#check Nat.pred_succ
```

When an operation is written as infix, the theorem names follow suit. For example, we write neg_mul_neg rather than `mul_neg_neg` to describe the pattern `-a * -b`.

Sometimes, to disambiguate the name of theorem or better convey the intended reference, it is necessary to describe some of the hypotheses. The word "of" is used to separate these hypotheses:

```
import Mathlib.Algebra.Order.Monoid.Lemmas

open Nat

#check lt_of_succ_le
#check lt_of_not_ge
#check lt_of_le_of_ne
#check add_lt_add_of_lt_of_le
```

The hypotheses are listed in the order they appear, *not* reverse order. For example, the theorem `A → B → C` would be named `C_of_A_of_B`.

Sometimes abbreviations or alternative descriptions are easier to work with. For example, we use `pos`, `neg`, `nonpos`, `nonneg` rather than `zero_lt`, `lt_zero`, `le_zero`, and `zero_le`.

```
import Mathlib.Algebra.Order.Monoid.Lemmas
import Mathlib.Algebra.Order.Ring.Lemmas

open Nat

#check mul_pos
#check mul_nonpos_of_nonneg_of_nonpos
#check add_lt_of_lt_of_nonpos
#check add_lt_of_nonpos_of_lt
```

These conventions are not perfect. They cannot distinguish compound expressions up to associativity, or repeated occurrences in a pattern. For that, we make do as best we can. For example, `a + b - b = a` could be named either `add_sub_self` or `add_sub_cancel`.

Sometimes the word "left" or "right" is helpful to describe variants of a theorem.

```
import Mathlib.Algebra.Order.Monoid.Lemmas
import Mathlib.Algebra.Order.Ring.Lemmas

open Nat

#check add_le_add_left
#check add_le_add_right
#check le_of_mul_le_mul_left
#check le_of_mul_le_mul_right
```

When referring to a namespaced definition in a lemma name not in the same namespace, the definition should have its namespace removed. If the definition name is unambiguous without its namespace, it can be used as is. Else, the namespace is prepended back to it in `lowerCamelCase`. This is to ensure that _-separated strings in a lemma name correspond to a definition name or connective.

```
import Mathlib.Data.Int.Cast.Basic
import Mathlib.Data.Nat.Cast.Basic
import Mathlib.Topology.Constructions

#check Prod.fst
#check continuous_fst

#check Nat.cast
#check map_natCast
#check Int.cast_natCast
```

Naming of structural lemmas

We are trying to standardize certain naming patterns for structural lemmas.

Extensionality

A lemma of the form `(∀ x, f x = g x) → f = g` should be named `.ext`, and labelled with the `@[ext]` attribute. Often this type of lemma can be generated automatically by putting the `@[ext]`

attribute on a structure. (However an automatically generated lemma will always be written in terms of the structure projections, and often there is a better statement, e.g. using coercions, that should be written by hand then marked with `@[ext]`.)

A lemma of the form `f = g ↔ ∀ x, f x = g x` should be named `.ext_iff`.

Injectivity #

Where possible, injectivity lemmas should be written in terms of an `Function.Injective f` conclusion which use the full word `injective`, typically as `f_injective`. The form `injective_f` still appears often in mathlib.

In addition to these, a variant should usually be provided as a bidirectional implication, e.g. as `f x = f y ↔ x = y`, which can be obtained from `Function.Injective.eq_iff`. Such lemmas should be named `f_inj` (although if they are in an appropriate namespace `.inj` is good too). Bidirectional injectivity lemmas are often good candidates for `@[simp]`. There are still many unidirectional implications named `inj` in mathlib, and it is reasonable to update and replace these as you come across them.

Note however that constructors for inductive types have automatically generated unidirectional implications, named `.inj`, and there is no intention to change this. When such an automatically generated lemma already exists, and a bidirectional lemma is needed, it may be named `.inj_iff`.

An injectivity lemma that uses "left" or "right" should refer to the argument that "changes". For example, a lemma with the statement `a - b = a - c ↔ b = c` could be called `sub_right_inj`.

Induction and recursion principles #

Induction/recursion principles are ways to construct data or proofs for all elements of some type T, by providing ways to construct this data or proof in more constrained specific contexts. These principles should be phrased to accept a `motive` argument, which declares what property we are proving or what data we are constructing for all T. When the motive eliminates into `Prop`, it is an induction principle, and the name should contain `induction`. On the other hand, when the motive eliminates into `Sort u` or `Type u`, it is a recursive principle, and the name should contain `rec` instead.

Additionally, the name should contain on iff in the argument order, the value comes before the constructions.

The following table summarizes these naming conventions:

motive eliminates into:	**`Prop`**	**`Sort u` or `Type u`**
value first	`T.induction_on`	`T.recOn`
constructions first	`T.induction`	`T.rec`

Variation on these names are acceptable when necessary (e.g. for disambiguation).

Predicates as suffixes #

Most predicates should be added as prefixes. Eg `IsClosed (Icc a b)` should be called `isClosed_Icc`, not `Icc_isClosed`.

Some widely used predicates don't follow this rule. Those are the predicates that are analogous to an atom already suffixed by the naming convention. Here is a non-exhaustive list:

- We use `_inj` for `f a = f b ↔ a = b`, so we also use `_injective` for `Injective f`, `_surjective` for `Surjective f`, `_bijective` for `Bijective f`...
- We use `_mono` for `a ≤ b → f a ≤ f b` and `_anti` for `a ≤ b → f b ≤ f a`, so we also use

`_monotone` for `Monotone f`, `_antitone` for `Antitone f`, `_strictMono` for `StrictMono f`, `_strictAnti` for `StrictAnti f`, etc...

Predicates as suffixes can be preceded by either `_left` or `_right` to signify that a binary operation is left- or right-monotone. For example, `mul_left_monotone : Monotone (· * a)` proves left-monotonicity of multiplication and not monotonicity of left-multiplication.

Prop-valued classes #

Mathlib has many `Prop`-valued classes and other definitions. For example "let be a topological ring" is written `variable (R : Type*) [Ring R] [TopologicalSpace R] [IsTopologicalRing R]` and "let be a group and let be a normal subgroup" is written `variable (G : Type*) [Group G] (H : Subgroup G) [Normal H]`. Here `IsTopologicalRing R` and `Normal H` are not extra data, but are extra assumptions on data we have already.

Mathlib currently strives towards the following naming convention for these `Prop`-valued classes. If the class is a noun then its name should begin with `Is`. If however is it an adjective then its name does not need to begin with an `Is`. So for example `IsNormal` would be acceptable for the "normal subgroup" typeclass, but `Normal` is also fine; we might say "assume the subgroup `H` is normal" in informal language. However `IsTopologicalRing` is preferred for the "topological ring" typeclass, as we do not say "assume the ring `R` is topological" informally.

Unexpanded and expanded forms of functions #

The multiplication of two functions `f` and `g` can be denoted equivalently as `f * g` or `fun x ↦ f x * g x`. These expressions are definitionally equal, but not syntactically (and they don't share the same key in indexing trees), which means that tools like `rw`, `fun_prop` or `apply?` will not use a theorem with one form on an expression with the other form. Therefore, it is sometimes convenient to have variants of the statements using the two forms. If one needs to distinguish between them, statements involving the first unexpanded form are written using just `mul`, while statements using the second expanded form should instead use `fun_mul`. If there is no need to disambiguate because a lemma is given using only the expanded form, the prefix `fun_` is not required.

For instance, the fact that the multiplication of two continuous functions is continuous is

```
theorem Continuous.fun_mul (hf : Continuous f) (hg : Continuous g) : Continuous
fun x ↦ f x * g x
```

and

```
theorem Continuous.mul (hf : Continuous f) (hg : Continuous g) : Continuous (f
* g)
```

Both theorems deserve tagging with the `fun_prop` attribute.

The same goes for addition, subtraction, negation, powers and compositions of functions.